# Project Direction Review

## 1. Current Direction

Not everyone has a team to help lift and move things, be it alone in a small workshop or a hobbyist looking to unpack a new tool. Our idea is to make a roof/suspended gantry system with an expandable frame that can fit a variety of workspaces. Not only would the gantry act as a kind of pack mule shuffling objects around the workspace, but depending on the attachment equipped, the system could also be adapted for different tasks. The goal is to make the system flexible enough to support both manual user control and additional automated functions as the project develops.

## 2. How Computer Science is Involved

- **Sensor communication and processing:** Collect and process information from position, distance, motion, limit, and payload-weight sensors. The software must use this data to determine the system's current position, load, and operating condition.
- **User interfaces and communication:** Develop interfaces for physical controllers, control panels, computers, and mobile devices. These interfaces must send commands, display system status, select attachments or operating modes, and report errors.
- **Simulation and virtual testing:** Create a virtual model of the gantry to test movement algorithms, sensor behavior, attachments, and possible collisions or failure cases before operating the physical prototype.
- **Diagnostics and safe shutdown:** Implement a real-time system that continuously collects and validates telemetry from motors, sensors, communication interfaces, and power-management components. It shall monitor the current health state of the components, detect abnormal conditions and communication failures. It will record relevant telemetry and events, then notify the user when fault conditions occur. If a component fails or an unsafe condition occurs, the system must report the problem and enter a safe shutdown state.
- **Pathfinding:** Would use the A* Pathfinding over Greedy or Dijkstra Pathfinding so it is not fully exploring every grid as it will be biased towards finding the goal. This would get turned into simplified waypoints compared to the A* raw Pathfind. This will use the Bresenham pathway to calculate a straight line on a step-like grid vs the pixel-like grid used by A*. In coordination with this would use an S curve motion to accelerate smoothly and have it slow down before arriving at the final point. This grid can get marked to tell the system if there are obstacles in the way before planning the path so it can avoid the obstacles. Potentially add multipoint stops which would use a Nearest Neighbor algorithm from traveling salesmen problems.
- **Software Integration:** Select and integrate the required microcontrollers, motor drivers, encoders, communication modules, and emergency-stop hardware. Software drivers and communication protocols will allow these components to exchange data and operate together reliably.

## 3. What Changes Were Made

Thursday's discussion helped us make the CS side of the project more specific. Before, a lot of our ideas were written as a list of technologies or features we could add. During the discussion, we focused more on what information the system actually needs while it's running, what the software has to decide and what should happen when something goes wrong.

We identified information the system may need to keep track of, including the size of the frame, payload weight, position relative to the frame, which attachment is connected, current motor movement, attachment data and user input. We also thought more about diagnostics and how the system should respond to different problems. For example, if two motors that should be moving together become unsynchronized, the system should be able to notice the difference through feedback such as motor encoders, pause user inputs, alert the user and resynchronize/recalibrate before continuing.

We also started looking more closely at how the different parts of the system will communicate. We discussed options such as CAN, SPI, $I^2C$ and UART, but we still need to compare them and decide what makes the most sense for our design.

## 4. Questions we are looking to answer

**Open Questions: What are the main things your team still needs to investigate or decide?**

- Which attachment or small group of attachments should we support in the first prototype?
- What should we use to build the simulation and what parts of the gantry need to be simulated before we start integrating the physical system?
- Which features are necessary for the first prototype and which ones should be saved as stretch goals?
- What sensors and feedback will we actually have available for things like position, motor synchronization, payload weight, frame stability, workspace limits and unsafe conditions?
- What communication protocols or combination of protocols makes the most sense for the different parts of the system?
- Which conditions should only warn the user and which conditions should immediately pause or stop the system?
- How would we fix a load that is swinging while in movement?